PERTEMUAN 11 · Praktik terbimbing

LR Parsing: Canonical LR, SLR, LALR, dan Konflik Parsing

Universitas Bale Bandung

Fakultas Teknologi Informasi · Program Studi Teknik Informatika

Mata Kuliah Teknik Kompilasi (MWCA017) · Semester Ganjil · 3 SKS

Dosen: Cecep Suwanda, S.Si., M.Kom.

Capaian Pertemuan

CPMK induk: CPMK 04 — Mengimplementasikan parser sederhana untuk validasi kode.

Sub-CPMK yang dinilai

Sub-CPMK 4.2: Membuat tabel aksi dan *goto* untuk parser SLR/LALR, lalu menguji dan men-*debug* konflik *shift-reduce* atau *reduce-reduce* yang muncul.

Artefak yang diselesaikan

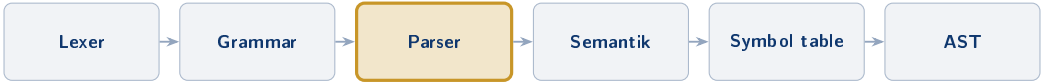
Tabel ACTION/GOTO SLR dan LALR, catatan konflik beserta resolusinya, log sebelum/sesudah, dan hasil uji string valid serta tidak valid.

Agenda Pertemuan (150 menit)

Menit	Kegiatan
25	Ceramah: item, closure, goto, dan koleksi kanonik
30	Latihan: automaton LR(0) dan tabel SLR
30	Lokakarya: penggabungan inti LR(1) dan tabel LALR
30	Praktik: membaca dan men-*debug* konflik
25	Perbandingan LL/LR dan pemeriksaan artefak
10	Penutup dan penugasan

Porsi praktik dan latihan 90 menit; ceramah 25 menit. Tabel dan log konflik dikumpulkan pada akhir pertemuan.

Posisi Pertemuan pada Rantai Proyek



- Pertemuan ini menyelesaikan simpul **parser** pada sisi bottom-up: item, automaton, dan tabel ACTION/GOTO.
- Tabel yang dibangun menjadi dasar parser Bison pada Pertemuan 12.
- Konflik yang berhasil di-*debug* menjadi bukti Sub-CPMK 4.2.

Item LR(0)

Definisi

Item LR(0) adalah aturan produksi dengan sebuah titik yang menunjukkan posisi parser. $[A \to \alpha \cdot \beta]$ berarti α sudah dikenali dan β diharapkan.

Aturan $E \to E + T$ memiliki empat item:

$$[E \to \cdot E + T], \quad [E \to E \cdot + T], \quad [E \to E + \cdot T], \quad [E \to E + T \cdot].$$

- Titik di awal: belum ada yang dikenali.
- Titik di akhir: ruas kanan sudah dikenali dan siap direduksi.

Closure

Aturan

Jika $[A \to \alpha \cdot B\beta]$ ada dalam himpunan item dan B nonterminal, tambahkan semua aturan $B \to \gamma$ sebagai $[B \to \cdot\gamma]$.

Contoh

Item $[E \to \cdot T]$ dengan aturan $T \to T * F \mid F$ menambahkan $[T \to \cdot T * F]$ dan $[T \to \cdot F]$.
Karena F juga nonterminal dengan $F \to (E) \mid \texttt{id}$, closure menambahkan $[F \to \cdot(E)]$ dan $[F \to \cdot\texttt{id}]$.

Closure diulang sampai tidak ada item baru.

Goto dan Koleksi Kanonik

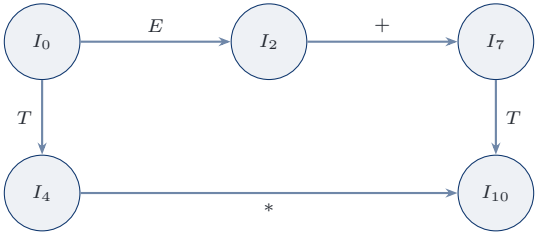
Goto

goto(I, X) adalah closure dari item yang titiknya bergeser melewati X. Jika I memuat $[A \to \alpha \cdot X\beta]$, hasilnya memuat $[A \to \alpha X \cdot \beta]$.

1. Mulai dari closure item awal $[S' \to \cdot S]$.
2. Untuk setiap state dan simbol X, hitung goto(I, X).
3. Jika hasilnya belum ada, tambahkan sebagai state baru.
4. Ulangi sampai tidak ada state baru.

Untuk grammar ekspresi pada `grammar-lr.bnf`, koleksi kanonik memiliki 12 state.

Automaton LR(0)



State awal adalah closure item awal; state penerima memuat $[S' \to S\cdot]$. Setiap transisi diberi label simbol grammar.

Konstruksi Tabel SLR(1)

1. Item $[A \to \alpha \cdot a\beta]$ dengan a terminal: isi ACTION$[I, a] =$ shift goto(I, a).
2. Item $[A \to \alpha\cdot]$ dan $A \neq S'$: untuk setiap $a \in$ FOLLOW(A), isi ACTION$[I, a] =$ reduce $A \to \alpha$.
3. Item $[S' \to S\cdot]$: isi ACTION$[I, \$] =$ accept.
4. Untuk nonterminal A, isi GOTO$[I, A] =$ goto(I, A).

Kunci SLR

SLR memakai automaton LR(0) dan himpunan FOLLOW secara global untuk memutuskan reduksi.

Potongan Tabel SLR(1)

State	id	+	*	$
0	s5			
1				
2		s7		acc
3		$r(T \to F)$	$r(T \to F)$	$r(T \to F)$
4		$r(E \to T)$	s8	$r(E \to T)$
5		$r(F \to \texttt{id})$	$r(F \to \texttt{id})$	$r(F \to \texttt{id})$

Sel kosong berarti kesalahan. s = shift, r = reduce, acc = terima. Tabel lengkap harus memuat semua state dan kolom GOTO untuk nonterminal.

Tabel ACTION/GOTO SLR (Artefak Sub-CPMK 4.2)

	ACTION			GOTO	
State	a	b	$\$$	S	A
0	s3	s4		1	2
1			acc		
2			$r(S \to A)$		
3	s3	s4			5
4			$r(A \to b)$		
5			$r(A \to aA)$		

Untuk grammar $S \to A,\ A \to aA \mid b$. Tabel lengkap memudahkan pemeriksaan konflik: sel dengan dua aksi adalah konflik yang harus diselesaikan.

Item LR(1) dan Closure LR(1)

Item LR(1)

$[A \to \alpha \cdot \beta,\ a]$, dengan a terminal atau \$. Lookahead menunjukkan token yang boleh muncul setelah reduksi.

Closure LR(1)

Jika $[A \to \alpha \cdot B\beta,\ a]$ ada, untuk setiap aturan $B \to \gamma$ dan setiap $b \in \text{FIRST}(\beta a)$, tambahkan $[B \to \cdot\gamma,\ b]$. Lookahead dihitung dari βa, bukan hanya FOLLOW(B).

LR(1) lebih presisi daripada SLR(1), tetapi jumlah state-nya jauh lebih besar.

Canonical LR(1) dan LALR(1)

Canonical LR(1)

Koleksi kanonik LR(1) dibangun seperti LR(0), tetapi state memuat item dengan lookahead. Ledakan state dapat menjadi masalah pada grammar bahasa nyata.

LALR(1)

LALR menggabungkan state LR(1) yang memiliki inti (*core*) sama, yaitu himpunan item tanpa lookahead.

1. Bangun koleksi kanonik LR(1).
2. Gabungkan state dengan inti sama.
3. Gabungkan lookahead item yang bersesuaian.
4. Isi ACTION dan GOTO dari item gabungan.
5. Periksa konflik.

Perbandingan SLR, LR(1), dan LALR(1)

Aspek	SLR(1)	LR(1)	LALR(1)
Lookahead	FOLLOW global	Per item	Per inti state
Jumlah state	Kecil	Besar	Sedang
Kekuatan	Paling lemah	Paling kuat	Hampir LR(1)
Konflik	Lebih mungkin	Paling sedikit	Sedikit
Praktik	Jarang	Jarang	Umum (Bison/Yacc)

LALR memberi keseimbangan antara kekuatan LR(1) dan ukuran tabel yang praktis.

Tabel ACTION/GOTO LALR

State LALR	Inti item	Lookahead	Aksi
I_2	$\{S \to A\cdot\}$	$\{\$\}$	reduce $S \to A$
I_4	$\{A \to b\cdot\}$	$\{\$\}$	reduce $A \to b$
I_5	$\{A \to aA\cdot\}$	$\{\$\}$	reduce $A \to aA$

Catatan

Untuk $S \to A,\ A \to aA \mid b$, penggabungan inti tidak menimbulkan konflik baru, sehingga tabel LALR sama dengan tabel SLR. Bison dan Yacc secara default membangun tabel LALR(1).

Konflik Shift-Reduce

Definisi

Satu sel tabel memiliki aksi shift dan reduce sekaligus.

Dangling else

stmt $\rightarrow$ `if` E `then stmt` | `if` E `then stmt else stmt`.
Setelah `if E then stmt` dan token `else`, parser dapat menggeser `else` atau mengurangi `stmt` terlebih dahulu.

Bison biasanya memilih shift; pilihan ini sesuai dengan aturan *else* terdekat, tetapi harus didokumentasikan.

Konflik Reduce-Reduce

Definisi

Dua aturan dapat direduksi pada sel yang sama.

Contoh

$A \to x$ dan $B \to x$ dengan FOLLOW yang tumpang tindih. Parser tidak dapat menentukan apakah x harus direduksi menjadi A atau B.

- Biasanya menandakan grammar terlalu ambigu atau aturan tumpang tindih.
- Selesaikan dengan menggabungkan aturan, *left factoring*, atau mengubah grammar.

Resolusi Konflik dengan Precedence

- Ubah grammar agar tidak ambigu.
- Tambahkan precedence dan associativity.
- Pilih aksi default secara eksplisit.
- Pisahkan aturan yang tumpang tindih.

Jangan menyembunyikan konflik dengan menaikkan kapasitas parser.

```
%left OR
%left AND
%left EQ NE
%left LT LE GT GE
%left PLUS MINUS
%left STAR SLASH PERCENT
%right NOT UMINUS
```

Listing 1: Deklarasi precedence pada Bison

Alur Debug Konflik

Jenis	Penyebab	Tindakan	Verifikasi
Shift-reduce	Precedence, associativity, atau *dangling else*	Deklarasi precedence, restrukturisasi, atau pilih aksi eksplisit	Konflik hilang; hasil sesuai aturan
Reduce-reduce	Dua aturan dengan ruas kanan sama atau FOLLOW tumpang tindih	Gabungkan aturan, *left factoring*, atau ubah grammar	Hanya satu reduce pada sel
Konflik setelah merge	Penggabungan inti LR(1) menjadi LALR	Restrukturisasi atau gunakan canonical LR(1)	Tabel LALR konsisten; string uji valid

Baca laporan: state, token, dan aksi yang bertabrakan. Terapkan satu resolusi pada satu waktu; simpan log sebelum/sesudah.

LL(k) versus LR(k)

Aspek	LL(k)	LR(k)
Arah	Kiri ke kanan, derivasi kiri	Kiri ke kanan, derivasi kanan
Lookahead	k token	k token
Left recursion	Tidak didukung	Didukung
Grammar	Harus difaktorkan	Lebih alami
Tabel	Relatif kecil	Lebih besar
Kesalahan	Mudah dilokalisasi	Perlu strategi tambahan
Implementasi	Recursive descent	Stack dan tabel

LL cocok untuk parser tulisan tangan dan diagnostik; LR cocok untuk grammar besar dan parser generator.

Kegiatan Mahasiswa di Kelas: P3

Tabel SLR/LALR dan *debug* konflik

1. Bangun koleksi kanonik LR(0) untuk `spec/grammar-lr.bnf`.
2. Isi tabel ACTION/GOTO SLR.
3. Bangun LR(1), gabungkan inti, lalu isi tabel LALR.
4. Bandingkan dengan alat laboratorium dan Bison.
5. Identifikasi konflik; catat state dan token pemicu.
6. Debug satu konflik pada satu waktu.
7. Uji string valid, tidak valid, dan kasus dangling else.

Tugas dan Artefak

Artefak Sub-CPMK 4.2

Tabel ACTION/GOTO SLR dan LALR yang lengkap, catatan konflik *shift-reduce* atau *reduce-reduce* beserta resolusinya, dan hasil uji.

- Tabel SLR lengkap: satu baris per state, ACTION untuk terminal, GOTO untuk nonterminal.
- Tabel LALR lengkap dengan inti dan lookahead.
- Tidak ada konflik tak terduga; konflik tersisa memiliki resolusi eksplisit.
- String valid diterima dan string tidak valid ditolak.
- Log sebelum/sesudah dan laporan singkat diserahkan.

Artefak ini menjadi bagian Praktikum 3 (12%) yang dilaporkan pada Pertemuan 12.

Rangkuman dan Jembatan

- Item LR(0) menandai posisi parser; closure dan goto membentuk state.
- Koleksi kanonik LR(0) menjadi dasar automaton dan tabel SLR(1).
- LR(1) menambahkan lookahead per item; LALR menggabungkan state dengan inti sama.
- Konflik shift-reduce dan reduce-reduce diselesaikan dengan precedence, restrukturisasi, atau resolusi eksplisit.
- LL(k) dan LR(k) memiliki trade-off; pilihan mengikuti kebutuhan grammar dan diagnostik.

Pertemuan berikutnya

Pertemuan 12 memakai Bison untuk membangun parser Subset-C dengan aksi semantik dan AST. Tabel LALR yang dipelajari hari ini adalah mesin di balik *yyparse*.